

TP Temps Réel - Conception d'un ECU

Régulateur de vitesse

Jean-Loup Chrestien de Beauminy

Dolovan Akdag

16 juin 2026

Support : ESP32 (dual core) + FreeRTOS, sous ESP-IDF. Code dans le dossier `ecu/`.

Table des matières

1	Choix d'architecture	2
2	Tableau des tâches	2
3	Diagramme des tâches et événements	2
4	Justification des mécanismes FreeRTOS	3
5	Charge cpu théorique	3
6	Priorité de l'uart par rapport au contrôle moteur	4
7	Choix des priorités et préemption	4
8	Monitoring et debug	4
9	Analyse de robustesse	5
9.1	Test de la couche protocole sur pc	5
9.2	Test du firmware complet sous émulation QEMU	6
10	Limitations	6

1 Choix d'architecture

L'idée de départ vient du cours : une tâche = une responsabilité. Plutôt qu'une grosse boucle qui lit l'uart, calcule le pid et émet tout au même endroit (ce qui serait monolithique et pénalisé), on a découpé le système en quatre tâches qui communiquent uniquement par des primitives FreeRTOS, jamais par des variables globales partagées à nu.

Les quatre tâches :

- **rx** : lit le bus uart et décode les trames (machine à états octet par octet). c'est le seul point d'entrée des données.
- **control** : le régulateur pid, réveillé toutes les 100 ms. il consomme les consignes et vitesses décodées par rx et émet la commande moteur.
- **stats** : la télémétrie, une fois par seconde. lit les compteurs et émet la trame stats.
- **failsafe** : la sécurité. normalement endormie, réveillée par un événement (front gpio ou timeout watchdog).

Le point clé de l'archi c'est l'isolation sur les deux cœurs de l'esp32. Le parser (rx) est épinglé sur le **core 0**, et tout ce qui est critique pour la régulation (pid, failsafe, stats) sur le **core 1**. Comme ça un flood de messages sur l'uart sature au pire le core 0, mais le pid sur le core 1 continue de tourner à l'heure. C'est la réponse directe à l'exigence non fonctionnelle 02 (isolation des défaillances).

La communication rx vers control passe par une **queue** de commandes. rx pousse des couples (**type**, **valeur**), control les dépile au début de chaque cycle. On a donc un producteur / consommateur classique : rx ne touche jamais aux variables internes du pid, et control n'a pas à savoir comment une trame est décodée. Si la queue est pleine (control en retard, ne devrait pas arriver) on jette la commande et on incrémente un compteur, on ne bloque pas rx.

L'état de supervision (mode courant, flag failsafe) est lu par control et écrit par rx (changement de mode) et par failsafe. Ces accès sont très courts donc protégés par une section critique (spinlock), pas par un mutex : on parle de quelques instructions, prendre un mutex coûterait plus cher que le travail lui même.

2 Tableau des tâches

nom	prio	cœur	périodicité	stack (octets)
failsafe	4	1	événementielle	2048
control	3	1	périodique 100 ms	2560
rx	2	0	bloquante sur uart	3072
stats	1	1	périodique 1000 ms	3072

Les priorités vont de 1 (idle+1) à 4. failsafe est la plus haute pour garantir son temps de réponse, stats la plus basse car non critique. Les tailles de stack sont dimensionnées à la louche puis vérifiables avec `uxTaskGetStackHighWaterMark`. rx et stats ont un peu plus de marge parce qu'elles font des logs et manipulent des buffers.

3 Diagramme des tâches et événements

Événements possibles :

- arrivée d'octets sur l'uart : rx débloquent, décode, éventuellement push une commande dans la queue et réarme le watchdog.
- tick 100 ms : control se réveille (`vTaskDelayUntil`), dépile la queue, calcule le pid, émet output (uniquement en mode auto).
- tick 1 s : stats émet les compteurs.
- 2 s sans trame valide : le timer expire, son callback notifie failsafe.

- front descendant sur le gpio d'erreur : l'isr notifie failsafe.
- réception d'un `mode_set` : rx remet le mode et efface le flag failsafe (reprise, exigence 06).

La justification des priorités est détaillée aux sections 6 et 7.

4 Justification des mécanismes FreeRTOS

- **queue** pour rx vers control : découple le producteur du consommateur et lisse les rafales. control lit à son rythme, rx n'est jamais bloqué par le pid. envoi non bloquant (timeout 0) pour ne pas ralentir le parser si la queue se remplit.
- **mutex** sur l'émission uart : trois tâches peuvent émettre (control, stats, failsafe). sans protection, deux écritures pourraient entrelacer leurs octets et casser une trame (interdit par nf03). le mutex sérialise l'émission. on a pris un mutex et pas un sémaphore binaire exactement pour l'**héritage de priorité** : si stats (p1) tient le mutex et que failsafe (p4) le demande, stats est temporairement remontée à p4 le temps de libérer, donc pas d'inversion de priorité non maîtrisée (nf04).
- **section critique** / **spinlock** pour les compteurs et l'état de supervision : accès de quelques cycles, sur un dual core il faut un spinlock (pas juste masquer les interruptions) pour rester cohérent entre les deux cœurs. plus rapide qu'un mutex pour un incrément.
- **software timer** pour le watchdog rx : un timer one-shot de 2 s réarmé à chaque trame valide. s'il expire c'est qu'on n'a rien reçu, son callback notifie failsafe. pas besoin d'une tâche dédiée qui boucle.
- **task notification** pour réveiller failsafe : le moyen le plus rapide et le plus léger de signaler une tâche depuis une isr ou un callback (plus rapide qu'un sémaphore). la valeur de la notification transporte la cause (gpio ou timeout).
- **vTaskDelayUntil** pour control et stats : maintient une période absolue sans dérive. un simple `vTaskDelay` accumulerait le temps de calcul à chaque tour et ferait dériver la période, donc du jitter. `vTaskDelayUntil` cale les réveils sur une base de temps fixe.

5 Charge cpu théorique

Le tick FreeRTOS est réglé à 1000 Hz (1 ms) dans `sdkconfig.defaults`. À 100 Hz par défaut la résolution serait de 10 ms, impossible de garantir un jitter sous 5 ms. À 1 ms le réveil du pid est calé au tick près.

Les wcet ci-dessous sont des estimations (pas de mesure oscillo, build only).

core 1 :

- control : dépiler la queue + calcul pid + construire et copier une trame output dans le ring buffer tx. estimé 200 us. charge = 200 us / 100 ms = **0,2 %**.
- stats : snapshot des compteurs + une trame + un log console. estimé 500 us toutes les secondes = **0,05 %**.
- failsafe : événementiel et rare, négligeable.

Total core 1 : largement sous **1 %**.

core 0 :

- rx : à 115200 bauds en 8n1 on a au plus 115200 / 10 = 11520 octets/s. chaque octet passe dans le parser (un switch, quelques instructions, environ 0,5 us). ça fait environ 6 ms de traitement par seconde, plus le coût des lectures uart et des interruptions du driver. on reste autour de **1 à 2 %** même en flood continu.

Conclusion : même sous surcharge uart maximale, le core 1 (la régulation) n'est quasiment pas chargé. On est très loin de la saturation, la marge est énorme. Le déterminisme du pid est garanti par construction.

6 Priorité de l'uart par rapport au contrôle moteur

C'est la question centrale du sujet. Si on traitait la réception uart en priorité haute, un flood de messages monopoliserait le cpu et retarderait le calcul du pid, donc la période de 100 ms ne serait plus tenue et le jitter exploserait.

On a fait l'inverse : **control (pid) est en priorité 3, strictement au dessus de rx (parser) en priorité 2**. Donc dès qu'un cycle pid doit s'exécuter, il préempte le parser, calcule, émet, et rend la main. Le flood attend. En plus, les deux tâches sont sur des cœurs différents (rx core 0, control core 1), donc en pratique elles ne se disputent même pas le cpu : le parser peut tourner à fond sur le core 0 sans jamais grignoter une microseconde au pid sur le core 1.

La réception n'est pas sacrifiée pour autant : la queue absorbe les rafales, et la perte éventuelle de messages sous flood extrême est comptée (compteur dropped) et non bloquante. On privilégie la régulation, qui est la fonction critique, et on dégrade proprement la réception si besoin.

7 Choix des priorités et préemption

L'ordonnanceur est en préemptif à priorité fixe (fpp). L'ordre des priorités suit la criticité temps réel :

1. **failsafe (4, max)** : la sécurité passe avant tout. comme elle est la plus prioritaire, dès qu'elle est notifiée elle préempte n'importe quelle autre tâche et s'exécute tout de suite. c'est ce qui garantit le temps de réponse sous 5 ms exigé : elle force output=0 et émet l'alarme sans attendre le prochain cycle du pid. le travail dans l'isr gpio est réduit au minimum (juste une notification, le reste est fait dans la tâche).
2. **control (3)** : la régulation, fonction principale, doit être prioritaire sur tout sauf la sécurité.
3. **rx (2)** : important mais ne doit pas perturber control.
4. **stats (1)** : télémétrie, peut attendre, aucune contrainte dure.

La préemption joue donc dans ce sens : failsafe > control > rx > stats. Un événement de sécurité interrompt un calcul pid en cours ; un calcul pid interrompt le parser ; le parser passe avant la télémétrie. C'est exactement l'ordre voulu pour respecter les contraintes.

8 Monitoring et debug

- **trame stats (0x83)** émise sur le bus toutes les secondes : rx valides, rx crc erreur, rx dropped, tx output, et un compteur en plus rx unknown (ids inconnus). l'outil de test côté pc peut suivre l'état de l'ecu en direct.
- **log console** sur l'uart0 (séparé du bus, qui est sur uart2) : chaque seconde la tâche stats sort une ligne avec le mode courant, le flag failsafe, tous les compteurs et le **heap libre** (`esp_get_free_heap_size`) pour détecter une fuite mémoire.
- **log de warning** à chaque failsafe avec la cause.
- **led** sur le gpio 2 (led onboard) : allumée en failsafe, éteinte dès la reprise. debug visuel immédiat sans brancher de console.
- séparer le bus binaire (uart2) des logs texte (uart0) évite de polluer les trames avec des messages de debug, et c'est lisible côté console.
- en debug plus poussé, `uxTaskGetStackHighWaterMark` permet de vérifier le dimensionnement des stacks et `vTaskGetRunTimeStats` de mesurer la charge réelle par tâche.

9 Analyse de robustesse

Le sujet décrit un script de stress qui injecte : des trames collées, des trames fragmentées, des crc faux, des ids inconnus, et un flood. La robustesse repose entièrement sur le parser, qui est une machine à états (start, len, data, crc) qui garde son état entre deux lectures uart.

- **trames fragmentées** : comme on décode octet par octet et qu'on conserve l'état du parser entre deux appels à la lecture uart, une trame coupée au milieu reprend exactement là où elle s'était arrêtée. pas de perte d'état.
- **trames collées** : après avoir traité une trame complète on repart directement à l'état start. l'octet suivant est aussitôt interprété comme le début de la trame d'après. on enchaîne sans rien perdre.
- **crc faux** : on recalcule le xor (len + type + payload) et on compare à l'octet reçu. si ça ne colle pas, la trame est jetée silencieusement, le compteur crc augmente et on resynchronise.
- **len incohérente** : si le len annoncé est nul ou dépasse la taille max acceptée, on rejette et on repart à start sans rien stocker. ça évite qu'une longueur absurde fasse déborder le buffer ou bloque le parser sur une trame qui n'arrivera jamais.
- **id inconnu** : trame structurellement valide mais type non géré, ignorée, le compteur unknown augmente (le crc était bon, donc on ne le compte pas en erreur crc). on ne plante pas, on continue.
- **flood** : le parser tourne sur le core 0 sans toucher au pid (core 1). la queue absorbe les rafales et, si elle sature, on droppe en comptant. la régulation reste à l'heure.

Côté sécurité, les deux déclencheurs de failsafe sont indépendants : si le pc arrête d'émettre, le watchdog de 2 s tombe et coupe ; un front sur le gpio d'erreur coupe aussi, en moins de 5 ms grâce à la tâche prioritaire. après, un simple mode_set en auto relance le tout.

9.1 Test de la couche protocole sur pc

On a testé à deux niveaux : d'abord un test unitaire de la couche protocole sur pc, puis le firmware complet sous émulation qemu avec le vrai script de stress. Pour le test unitaire on a isolé la couche protocole (construction de trame, crc, et la même machine à états que dans le firmware) et on l'a compilée et exécutée sur pc avec les mêmes cas que le script : trame propre, trames collées, trame fragmentée, crc cassé, id inconnu. La compilation passe avec `-Wall -Wextra -Werror` et le décodage se comporte comme prévu.

```
== test couche protocole ==

1. setpoint 90.0 -> trame de 9 octets : aa 05 00 01 00 00 b4 42 f2
   -> trame valide type=0x01 val=90.00

2. deux trames collees (setpoint + speed)
   -> trame valide type=0x01 val=90.00
   -> trame valide type=0x02 val=42.50

3. trame fragmentee (2 morceaux)
   -> trame valide type=0x02 val=42.50

4. trame avec crc casse
   -> crc faux (recu 0x0d attendu 0xf2) rejet

5. id inconnu (0x42)
   -> id inconnu 0x42 ignore

== bilan : ok=4 unknown=1 rejets=1 ==
```

Listing 1 – sortie réelle du test protocole (gcc, sur pc)

On retrouve bien les six comportements attendus : la trame valide est décodée, les deux trames collées sont séparées correctement, la trame fragmentée est reconstituée, le crc cassé est rejeté, et l'id inconnu est ignoré sans planter.

9.2 Test du firmware complet sous émulation QEMU

Faute de carte physique, on a fait tourner le firmware compilé (esp-idf v5.4.4) dans l'émulateur **qemu xtensa** fourni par espressif. Le bus ecu (uart2) est exposé par qemu sur un socket tcp, et le script de stress du sujet s'y connecte directement (au lieu d'un `/dev/ttyUSB0`). C'est donc le vrai scénario du sujet (régulation, flood, trames cassées, silence radio) rejoué de bout en bout, le firmware s'exécutant réellement.

Ce que la trace montre, point par point :

- **régulation** : en mode auto, `rxok` et `out` montent régulièrement, la commande moteur est émise à chaque cycle.
- **flood** : les 50 trames à id aléatoire sont comptées en `unk=50` (ids inconnus), sans planter et sans bloquer la régulation.
- **trames cassées** : les deux trames volontairement invalides (crc faux et fragment mal terminé) donnent `crc=2`.
- **aucune perte** : `drop=0` sur tout le run, la queue n'a jamais débordé même pendant le flood.
- **failsafe** : à la coupure des émissions, l'ecu passe en `fs=1`, force `output=0` et émet l'alarme. le script côté pc confirme (`verification reussie`).
- **pas de fuite mémoire** : `heap` reste à 289312 octets du début à la fin.

La seule chose que l'émulation ne valide pas finement, c'est le timing exact (qemu n'est pas cycle-accurate), donc le jitter sous 5 ms reste à confirmer à l'oscilloscope sur une vraie carte.

10 Limitations

- **pas de carte physique** : le firmware compile sous esp-idf v5.4.4 (`idf.py set-target esp32 && idf.py build`) et tourne sous émulation qemu, mais n'a pas été testé sur un vrai esp32. comme qemu n'est pas cycle-accurate, le **jitter sous 5 ms** et le temps de réaction du failsafe restent à mesurer à l'oscilloscope sur cible.
- du même coup les **wcet et la charge cpu restent des estimations**, pas des mesures hardware. à confirmer avec `vTaskGetRunTimeStats` et l'analyse du high water mark sur une vraie carte.
- les **coefficients du pid** (`kp=1`, `ki=0.1`, `kd=0.01`) sont ceux de l'exemple du sujet, ils ne sont pas réglés pour un vrai modèle de véhicule. il faudrait les ajuster sur le banc.
- le bus ecu est sur **uart2** (pins 16/17), séparé de la console. pour brancher le script de test il faut un adaptateur usb-série sur ces pins, pas le port usb de debug. (on peut basculer le bus sur `uart0` et couper les logs si on veut coller au script tel quel.)
- la trame **dbg (0xff)** n'est pas émise, on s'en tient aux messages exigés. elle pourrait servir à dumper l'état interne du pid pour le réglage.
- pas de persistance : au reset on repart en mode off, compteurs remis à zéro.

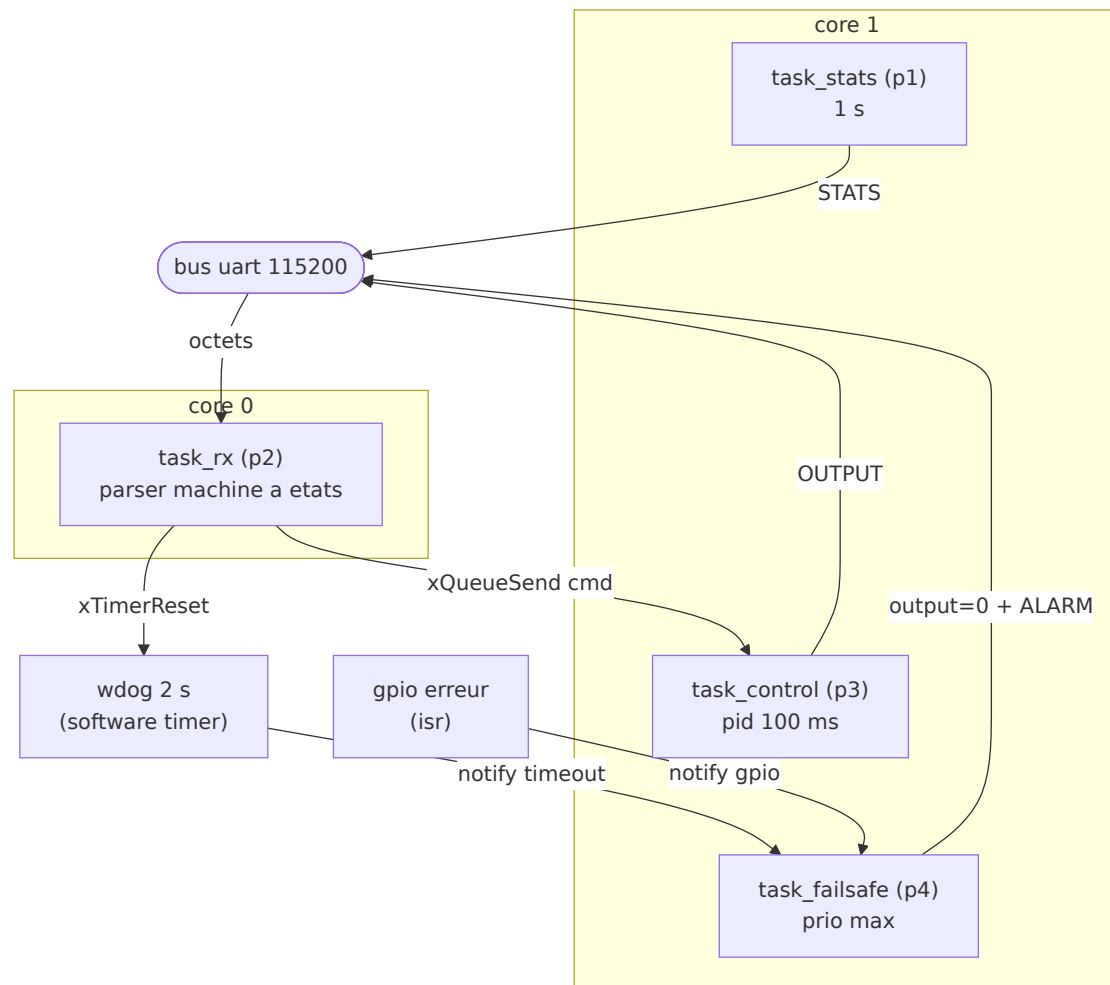


FIGURE 1 – tâches, cœurs et flux d'événements (primitives freertos en légende)

```

$ ECU_PORT=socket://localhost:5557 python tools/stress_test.py
--- debut phase normale (30s) ---
[SPEED] (31.60 km/h) | commande: 55.77
[SPEED] (44.38 km/h) | commande: 51.75
[TELEMETRIE] recue (1s)
[SPEED] (55.49 km/h) | commande: 47.50
[SPEED] (62.40 km/h) | commande: 35.08
[ ... regulation : la vitesse converge vers la consigne 90 ... ]
[SPEED] (86.92 km/h) | commande: 47.53
--- debut phase de stress ---
action: surcharge cpu (flood)...
action: envoi de trame fragmentee...
action: envoi erreur crc...
--- debut phase normale (5s) ---
[SPEED] (86.92 km/h) | commande: 47.96
[SPEED] (86.92 km/h) | commande: 0.00
[ALERTE ECU] : timeout rx
verification reussie : l'ecu a coupe la commande moteur (output=0)
test termine.

```

FIGURE 2 – côté pc : le script de stress branché sur l'ecu émulé. la vitesse converge vers la consigne, le stress est encaissé, puis le failsafe coupe la commande (output=0) avec l'alarme timeout rx

```

qemu-system-xtensa -M esp32 (console uart0 de l'ecu)
I (1904) ecu: demarrage ecu
I (2916) ecu: mode=0 fs=0 | rxok=0 crc=0 drop=0 out=0 unk=0 | heap=289312
W (3921) ecu: failsafe cause=1
[ ... le script passe en mode auto et regule ... ]
I (20912) ecu: mode=2 fs=0 | rxok=37 crc=0 drop=0 out=34 unk=0 | heap=289312
I (28912) ecu: mode=2 fs=0 | rxok=117 crc=0 drop=0 out=114 unk=0 | heap=289312
I (44913) ecu: mode=2 fs=0 | rxok=275 crc=0 drop=0 out=274 unk=0 | heap=289312
I (48913) ecu: mode=2 fs=0 | rxok=313 crc=2 drop=0 out=314 unk=50 | heap=289312
W (54849) ecu: failsafe cause=1
I (55913) ecu: mode=0 fs=1 | rxok=353 crc=2 drop=0 out=374 unk=50 | heap=289312

```

FIGURE 3 – côté ecu (console uart0) : les compteurs de télémétrie pendant le run